

ECHO SHIFT

A time-loop puzzle game — submitted for EVOX 1.0 (DETOX), "Build a Game"

Concept

Echo Shift is a browser-based puzzle game built around a single idea: your own past actions don't disappear when a loop ends — they keep happening. Every time a timed loop finishes, the run you just made is kept as a translucent **echo** that repeats those exact movements forever, tick for tick, while you start the next loop from the beginning with a clean slate.

The puzzles are built from that one rule. Certain doors only open when several pressure plates are held down at the same instant — something a single body cannot do. So the player has to plan across loops: one loop to place an echo on plate A, a second to place another echo on plate B, and a final loop where the live player reaches plate C while both echoes are still replaying their parts, all lining up on the same tick. The challenge is entirely about sequencing and timing, not reflexes.

Gameplay & progression

- **Level 1 — First Steps:** one plate, no echo needed. Pure movement tutorial.
- **Level 2 — Two Hands:** two plates, one door. Requires exactly one echo.
- **Level 3 — Triangulation:** three plates. Requires two stacked echoes plus the live player.
- **Level 4 — Branch Point:** four plates reached via a branching corridor. Requires three stacked echoes.
- **Level 5 — Two Doors:** four plates behind two independent, sequential locks — tests planning order, not just echo count.

Controls are arrow keys / WASD to move, **L** to end a loop early, and **R** to restart. A touch D-pad and on-screen buttons make it playable on mobile as well.

Approach

The mechanic was designed as an explicit rule set before any code was written: a fixed-length tick loop, a recorded direction per tick, an echo that replays its recording in lockstep with the same tick counter every loop, and doors that latch open the moment their required plate-ids are all simultaneously active. Levels were hand-authored as simple connected corridor/cross-shaped grids specifically because that shape is easy to verify by hand has no isolated, unreachable pockets.

Rather than trusting that by eye, every level was additionally checked with a small headless solver script that re-implements the same simulation and plays each level itself with real pathfinding — confirming each one is actually solvable within its loop budget before it was considered finished. That check caught a real sequencing bug on Level 5 during development (a plate that was assumed reachable immediately actually sits behind the first door).

Tech

Single self-contained HTML file — vanilla JavaScript and the Canvas API, no frameworks, no build step, no external runtime dependencies (only Google Fonts for typography). This keeps it trivial to host as a static page (GitHub Pages, Netlify, Vercel) and trivial for a judge to run locally by opening the file directly.

Team

Team size: 1–4 (per competition rules). Repository: github.com/priyanshu-sarjan/PS-G-Games-

Prompts used to build this game

Exact prompt log, as required by the submission guidelines

Prompt 1 — the brief (verbatim, as given to the AI)

The build began from a single prompt to Claude (Anthropic) that pasted the full official EVOX 1.0 brief and theme reveal in full — team size 1–4, the prompt-engineering / game-building competition format, the 22–27 Sep 2026 Unstop submission window, and the requirement for a live deployed link plus a single PDF (concept write-up, then exact prompts) — followed by this instruction:

“build a game as with in Claude AI free version limit tokens and push all the code to that given repo as <https://github.com/priyanshu-sarjan/PS-G-Games-.git> with well defined repo”

That single prompt set every constraint the build had to satisfy: an innovative game judged on Innovation, Game Creativity, Approach, Prompting and Engaging Uniqueness; a live link plus a two-part PDF; delivery as a proper GitHub repository rather than a loose file; and the whole build done inside one AI conversation, without external paid tooling.

Build plan carried out from that prompt

1. Pick a concept where the mechanic itself is the prompt-engineering payoff

Decided against a reskinned platformer or quiz in favor of a mechanic where the core loop is genuinely new: a time-loop puzzle in which the player's own recorded movement becomes an "echo" that keeps acting after the player restarts. This also produces a natural, reusable difficulty curve — more required echoes — without inventing new mechanics per level.

2. Design the simulation rules before any visuals

Wrote out the exact rule set: a fixed-length tick loop; a per-tick recording array; an echo that replays its recording in lockstep with the same global tick counter every loop; pressure plates active when any entity (player or echo) occupies them; and doors that latch open permanently the instant their required plate-id set is simultaneously active. Precise rules, not a vague description, are what make an emergent mechanic actually fair and solvable.

3. Hand-design 5 levels as ASCII grids, escalating one variable at a time

Level 1 (one plate, no echo) → Level 2 (two plates, one echo) → Level 3 (three plates, two echoes) → Level 4 (four plates via a branching corridor, three echoes) → Level 5 (two independent doors, four plates, tests sequencing). Every grid was deliberately built as a single connected corridor/cross shape, which is trivial to hand-verify for connectivity — a maze with an accidentally isolated pocket would silently make a level unsolvable.

4. Build the game as one dependency-free HTML file

Canvas-based rendering, a DOM-based HUD and overlays, keyboard and on-screen D-pad input, position interpolation for smooth motion on top of discrete tick logic, localStorage best-run tracking, and reduced-motion / mobile-safe-area handling. No frameworks or external code, so the file can be opened

directly or hosted anywhere as a static page.

5. Verify every level is solvable — don't just eyeball it

Wrote a headless Node script (test/sim.js) that re-implements the same tick / echo / door logic and plays each level itself using real breadth-first-search pathfinding, reporting how many loops it took to solve.

This caught a real bug on the first pass: Level 5's third plate sits behind a door that plates 1 and 2 must open first, which an upfront "is every plate reachable from the start" check flagged incorrectly as unsolvable. The check was corrected to verify reachability against live door state at the moment each plate is targeted, and all five levels then verified as solvable within their intended loop budgets.

6. Package it as a real, well-defined repository

README.md for setup, deployment and a mapping to the judging criteria; this document (PROMPTS.md / SUBMISSION.pdf) for the required prompt log; test/sim.js checked in as proof of the verification step; and game/index.html as the deliverable itself — structured as an actual small project rather than a single dropped file, per the brief's own "well defined repo" instruction.

Full prompt log and reasoning are also kept in PROMPTS.md in the repository root for reference.